AI-Driven Attack Defense for IoT Smart City in 6G Networks

Presented for

Private higher education institution for engineering and technology

Level

4th DATA

Prepared by :

Zahra Bendhaw
Tasnim Kheder
Mustapha Aziz Belkadhi
Anas Yahyaoui
Rami Klouz
Ahmed Karray

School year: 2025–2026

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

IOT: Internet Of Things

DDoS: Distributed Denial of Service

WiFi: Wireless Fidelity

MITM: Man-in-the-Middle

AI: Artificial Intelligence

ROI: Return on Investment

SLAs: service-level agreements

GENERAL INTRODUCTION

In a world where digital technologies are rapidly transforming industries and daily life, innovation has become a key driver of performance, competitiveness, and sustainability. Organizations today face increasing challenges related to efficiency, scalability, security, and decision-making, all within highly dynamic and data-driven environments. As a result, modern technological solutions are no longer optional they are essential.

This project is developed within this context of continuous technological evolution. It aims to address real-world challenges by leveraging contemporary tools, methodologies, and intelligent systems to deliver a practical and impactful solution. By combining theoretical knowledge with hands-on implementation, the project seeks to bridge the gap between academic learning and professional practice.

Beyond its technical dimension, this project reflects a broader vision: to design a solution that is reliable, adaptable, and aligned with current market and user needs. It emphasizes structured analysis, thoughtful design choices, and performance-oriented implementation, while also considering constraints such as cost, security, and usability.

Through this work, we demonstrate not only technical competence but also an understanding of how emerging technologies can influence decision-making, optimize processes, and create added value. This report presents the different phases of the project, from problem analysis and design to implementation and evaluation, highlighting the choices made and the results achieved.

Ultimately, this project illustrates how well-designed technological solutions can contribute to innovation and serve as a foundation for future improvements and real-world applications.

Chapter

1

General Context and Problem Statement

Contents

1.1 Scientific Context

The development of 5/6G networks will bring major improvements to smart cities. These networks will support many IoT devices, faster connections, and smarter services. However, using many different IoT devices creates new security risks. Traditional security methods are no longer enough to protect these networks.

Recent research shows that Artificial Intelligence (AI) is needed to improve network security. AI can help networks detect threats automatically and respond quickly. Machine Learning and Deep Learning are especially useful for analyzing network traffic, finding unusual activity, and identifying cyber-attacks. These AI methods work well in complex environments like smart cities with 6G networks.

For this reason, AI-based intrusion detection systems are an important research area. These systems can help make IoT networks more secure and reliable.

1.2 Problem formulation

Even though 6G networks offer many advantages for smart city applications, securing large-scale IoT systems remains a significant challenge. Smart city networks generate massive volumes of heterogeneous data that are transmitted at very high speeds, making network monitoring and attack detection difficult when using traditional security methods.

The main challenge lies in the early and accurate detection of cyber-attacks such as Distributed Denial of Service (DDoS), Man-in-the-Middle (MITM), and botnet attacks. These attacks are often hidden within normal network traffic, evolve continuously, and are difficult to distinguish from legitimate behavior. As a result, existing approaches may generate false alarms, fail to detect real attacks, or respond too slowly. Moreover, smart city environments require fast and scalable detection mechanisms that operate reliably at large scale.

Therefore, this project addresses the following research problem:

How can an intelligent system be designed to automatically analyze 6G IoT network traffic and accurately detect cyber-attacks in smart city environments?

To address this problem, Machine Learning and Deep Learning models are employed to learn patterns in network data and distinguish between normal and malicious activity. The objective is to develop a detection system that is fast, accurate, and reliable for large-scale smart city IoT networks.

1.3 Study of The Existing

Several smart city solutions are currently deployed worldwide to improve urban services through IoT and advanced communication networks. These solutions demonstrate how large-scale connected systems operate in real environments and highlight both their benefits and limitations, particularly in terms of security, scalability, and data management.

1.3.1 Advanced Smart City Platform with AI-Based IoT Security

Barcelona has deployed a smart city platform integrating thousands of IoT devices via high-speed networks (4G/5G) to manage traffic, energy, public safety, and environmental monitoring in real time. While this system improves urban efficiency and decision-making, it faces serious cybersecurity challenges: IoT devices can be hijacked for botnet or DDoS attacks, and unsecured communications risk MITM attacks. Traditional security methods struggle to handle the scale and diversity of traffic, highlighting the need for AI-driven intrusion detection to monitor, detect, and respond to anomalies automatically.[1]

1.3.2 Smart Traffic Management

Singapore uses smart traffic lights, surveillance cameras, and IoT sensors to reduce traffic congestion, detect accidents, and optimize traffic flow in real time. These intelligent systems enable better traffic coordination and improved urban mobility. However, they also face several limitations, including high infrastructure and deployment costs, cybersecurity risks related to connected cameras and sensors, potential traffic disruptions in the event of system failures, and the complexity of processing large volumes of real-time data.[2]

Land Transport Authority
We Keep Your World Moving

Figure 1.1: Smart Traffic Management System

1.4 Business Objectives

The main objective of this project is to improve the security and reliability of smart city IoT systems through intelligent cyber-attack detection and analysis.

The specific business objectives are:

Protect business continuity and operations: Safeguard your IoT infrastructure from disruptions that could halt production, interrupt services, or damage your brand reputation in the market.

Protect your bottom line: Avoid costly business interruptions, emergency response expenses, regulatory fines, legal liabilities, and lost revenue from operational downtime or compromised customer data.

Maximize security ROI (Return on Investment): Reduce overhead costs by automating threat detection and response, allowing your security team to focus on strategic initiatives rather than routine monitoring tasks.

By achieving these goals, this project will help create smart cities that are safer, smarter, and ready for the future, giving citizens confidence in the technology that surrounds them.

1.4.1 Stakeholders and End Users

The success of an AI-driven intrusion detection system for smart city IoT networks depends on the involvement of multiple stakeholders and end users. Each actor plays a critical role in the deployment, operation, and impact of the proposed solution.

6G Network Operators and Telecommunications Providers

Telecommunication operators managing 6G connectivity infrastructure play a central role in guaranteeing high-speed, low-latency communication between IoT devices. They depend on the intrusion detection system to monitor network traffic integrity, prevent large-scale disruptions, and maintain service-level agreements (SLAs)

End users of the proposed system primarily consist of security analysts who monitor IoT network traffic and investigate detected threats, as well as city operators responsible for critical services such as traffic management, energy distribution, and public safety. The system supports these users by providing timely alerts, accurate threat detection, and decision-support information, enabling rapid and informed responses to cyber incidents.

This identification of stakeholders and end users ensures that the proposed solution is aligned with real operational needs and delivers measurable value to smart city environments.

1.4.2 Business Objectives to Data Science Tasks Mapping

In order to ensure a strong alignment between business needs and technical implementation, this project establishes a clear mapping between business objectives and corresponding data science tasks. This mapping guarantees that each business requirement is directly supported by an appropriate analytical or machine learning approach, ensuring measurable and operational impact.

Table 1.1: Mapping between business objectives and data science tasks

Business Objective	**Data Science Objectives**
Protect business continuity and operations	Develop machine learning models to monitor IoT network traffic and device behavior in real time, enabling accurate detection, identification, and localization of cyber-attacks before they spread.
Protect your bottom line	Build predictive and anomaly detection models to minimize service downtime, reduce the impact of cyber incidents, and prioritize high-risk attacks that could cause infrastructure damage or data breaches.
Maximize security ROI (Return on Investment)	Design automated threat detection systems that reduce manual security monitoring, optimize alert accuracy, and improve efficiency while maximizing security ROI.

1.5 Data Science Tasks

This project applies the CRISP-DM methodology with a specific focus on **cyber-attack detection in 6G-enabled smart city IoT networks**. The data science objectives are designed to transform raw network traffic data into actionable security intelligence.

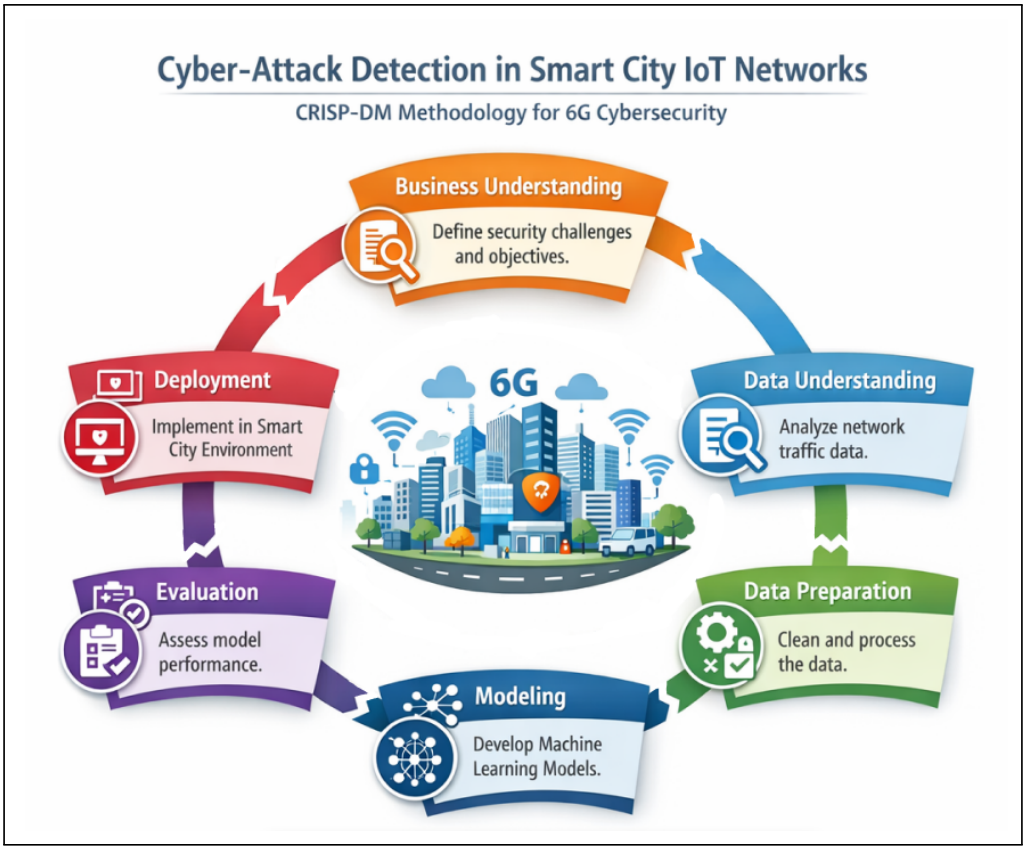

Figure 1.2: cyber-attack detection in smart city IoT networks

1.5.1 Business Understanding

The first step consists of translating smart city security challenges into a clear data science problem. The objective is to detect cyber-attacks within large and complex 6G IoT networks while minimizing false alarms and missed detections. This requires understanding how cyber threats impact critical smart city services and defining measurable goals that support automated, fast, and reliable intrusion detection at city scale.

1.5.2 Data Understanding

This phase aims to analyze and understand network traffic data generated by IoT devices in smart city environments. The objective is to identify the characteristics of normal and malicious traffic by studying flow-based features, traffic patterns, and protocol behaviors. Special attention is given to detecting data quality issues such as noise, missing values, class imbalance, and correlations, as well as evaluating whether the dataset realistically represents real-world smart city network conditions.

1.5.3 Data Preparation

The purpose of data preparation is to construct a high-quality dataset that improves the performance of cyber-attack detection models. This involves cleaning and normalizing network traffic features, engineering security-relevant variables that highlight abnormal behavior, addressing class imbalance between normal and attack traffic, and organizing the data into training, validation, and test sets. These steps ensure that the data accurately reflects both legitimate network activity and malicious behavior.

1.5.4 Modeling

The modeling phase focuses on developing and comparing Machine Learning and Deep Learning models specifically designed for intrusion detection in smart city IoT networks. The objective is to enable the models to learn complex and evolving patterns that distinguish normal traffic from cyber-attacks, while maintaining scalability and fast response times. Model optimization is performed to achieve a balance between detection accuracy, robustness, and computational efficiency.

1.5.5 Evaluation

The evaluation phase aims to assess the effectiveness of the developed models in a realistic security context. Performance is measured using appropriate metrics such as accuracy, precision, recall, F1-score, and ROC-AUC, with particular emphasis on reducing false positives and false negatives. The goal is to select a model that provides reliable and early detection of cyber-attacks while minimizing unnecessary alerts in smart city environments.

1.5.6 Deployment

The final phase focuses on deploying the selected intrusion detection model into a practical smart city monitoring environment. The objective is to enable near real-time analysis of IoT network traffic, provide clear visualization of detected threats, and support security operators in making rapid and informed decisions. This deployment demonstrates the real-world applicability of the proposed solution within large-scale smart city infrastructures.

1.6 Sustainable Development Goals

This project contributes to several United Nations Sustainable Development Goals by improving smart city security:

SDG 9: Industry, Innovation, and Infrastructure Develops AI-based security systems that protect critical infrastructure and enable cities to safely adopt new technologies for sustainable development.

SDG 11: Sustainable Cities and Communities Protects IoT systems that manage transportation, energy, water, and public services, allowing cities to expand smart services that improve urban living and resource management.

SDG 17: Partnerships for the Goals Contributes to global knowledge sharing in AI and cybersecurity through documented research that supports international collaboration between academia, government, and industry.

Figure 1.3: Sustainable Development Goals

1.7 Conclusion

This project addresses the critical challenge of securing 6G-enabled smart city IoT networks against cyber-attacks. By developing an AI-based intrusion detection system using Machine Learning and Deep Learning techniques, this work aims to provide accurate, fast, and scalable threat detection for next-generation urban infrastructure.

Following the CRISP-DM methodology, the project will systematically develop and evaluate multiple detection models to identify the most effective approach for distinguishing normal traffic from malicious activity. The resulting system will contribute to safer and more resilient smart cities while supporting multiple UN Sustainable Development Goals.

The expected outcomes include a validated detection framework, comparative analysis of AI techniques for IoT security, and a practical prototype that demonstrates real-world applicability for smart city operators and security teams.

Chapter 2

Data Understanding

Contents

2.1 Introduction

This chapter presents a comprehensive exploratory data analysis (EDA) of the network traffic datasets across four 5G/6G service categories: mMTC, URLLC, eMBB, and the aggregated g6 dataset. The primary objectives are to characterize class distributions and identify imbalance patterns that may bias detection models, assess data quality dimensions including missing values, duplicates, and outliers, examine feature correlations to detect multicollinearity and redundancy, and analyze data type compositions to inform preprocessing strategies.

Understanding these foundational properties is critical for designing robust intrusion detection systems, as network slicing scenarios exhibit heterogeneous traffic characteristics requiring slice-specific modeling considerations. The findings herein directly inform feature engineering decisions, sampling strategies, and model selection criteria developed in subsequent chapters.

2.2 Libraries and Tools Used

2.2.1 Programming Language:

Python 3.x — chosen for its rich ecosystem in data science and machine learning.

Figure 2.1: Python logo

2.2.2 Data Manipulation and Numerical Computing:

- **Pandas (pandas)** Used for loading, cleaning, transforming, and analyzing tabular data.
- **NumPy (numpy)** Used for numerical operations and efficient array computations.

2.2.3 Data Visualization:

- **Matplotlib (matplotlib.pyplot)** Used for plotting graphs and visualizing data distributions and model results.
- **Seaborn (seaborn)** Built on top of Matplotlib, used for statistical visualizations and improved aesthetics.

2.2.4 Data Preprocessing and Feature Engineering

Scikit-learn – Preprocessing

The following pre-processing techniques were used to prepare the data before training the models:

- **StandardScaler, RobustScaler**: normalisation of numerical variables to improve model convergence and reduce the influence of outliers.
- **OneHotEncoder**: encoding categorical variables as binary variables.
- **SimpleImputer, KNNImputer**: handling missing values using simple strategies (mean, median) and neighbourhood-based strategies.
- **FunctionTransformer**: applying custom transformations to features.

ColumnTransformer: application of separate pre-processing pipelines to numerical and categorical variables.

Pipeline: construction of complete processing flows ensuring the reproducibility of experiments and preventing data leakage.

2.2.5 Model Selection and Validation:

- **Train-Test Split (train**$_{t}est_{s}plit)Usedtoevaluatemodelgeneralization.$
- **Stratified K-Fold Cross-Validation (StratifiedKFold) Preserved class distribution during cross-validation.**
- **Cross-validation utilities (cross**$_{v}alidate)Usedtocomparemodelsusingmultipleevaluationmetrics.$

2.2.6 Machine Learning Models

Several classification models were implemented and compared in order to evaluate their performance:

- **Logistic Regression** : a linear classifier used as a baseline model.
- **K-Nearest Neighbors (KNN)** : a distance-based classification model relying on the similarity between data points.
- **Random Forest Classifier** : an ensemble model based on decision trees, providing improved robustness and reduced overfitting.
- **Gradient Boosting Classifier** : a boosted ensemble model capable of capturing complex patterns and improving predictive performance.

2.2.7 Model Evaluation and Interpretation

Model evaluation and interpretation were performed using several metrics and analysis tools:

a) Classification Metrics

- **classification_report** : provides precision, recall, F1-score, and support for each class.
- **confusion_matrix, ConfusionMatrixDisplay** : used to analyze the distribution of correct and incorrect predictions.

b) ROC and AUC Analysis

- **roc_auc_score** : measures the model's ability to distinguish between classes.
- **RocCurveDisplay** : enables visualization of the Receiver Operating Characteristic (ROC) curve.

c) Permutation Feature Importance

This method was used to analyze the contribution of individual features to model performance and to identify the most influential variables.

2.2.8 Project Organization

Project structure and file management were handled using the **Pathlib** library (`pathlib.Path`), enabling operating system–independent file and directory handling.

2.3 Data Collection

2.3.1 Dataset Sources

This project utilizes four complementary network traffic datasets designed for intrusion detection research in 5G/6G network slicing environments. The datasets correspond to three standardized 5G service categories—massive Machine-Type Communications (mMTC), Ultra-Reliable Low-Latency Communications (URLLC), and enhanced Mobile Broadband (eMBB)—along with an aggregated sixth-generation (g6) dataset combining multi-slice traffic scenarios.

These datasets were selected to evaluate model performance across diverse network slicing contexts characterized by distinct Quality-of-Service (QoS) requirements, traffic patterns, and attack surfaces. The individual slice datasets enable assessment of specialized detection capabilities, while the aggregated g6 dataset facilitates evaluation of generalized cross-slice intrusion detection.

2.3.2 Data Loading Procedure

The datasets are loaded from CSV files stored in the project directory. A robust loading function verifies file existence and handles missing files gracefully to ensure reproducibility across different environments.

```
Loaded mMTC: (4615, 52) from mMTC.csv
Loaded URLLC: (4033, 52) from URLLC.csv
Loaded eMBB: (5808, 52) from eMBB.csv
Loaded Global: (14456, 52) from Global.csv
Loaded g6: (10000, 13) from 6G_English_Education_Network_Traffic.csv
```

Figure 2.2: Data Loading

2.4 Data Description

2.4.1 Data Types Distribution

Understanding feature data types is essential for selecting appropriate encoding strategies and preprocessing pipelines.

Visualization: Horizontal bar charts showing count of features by data type per dataset.

Key Findings:

- **mMTC/eMBB/URLLC:** Float64 dominates (∼30–35 features), followed by int64 (∼12–13) and object (∼6–7)
- **g6:** Inverted pattern—object type majority (∼8), with fewer int64 (∼3) and float64 (∼2)

Data Type Remarks

Individual slices show **consistent numerical dominance** (float64/int64 ∼85% of features), suitable for direct statistical modeling. The prevalence of float64 suggests continuous network measurements (timestamps, packet sizes, rates).

Conversely, **g6's categorical skew** (∼50% object type) indicates fundamental structural differences from constituent slices. This inversion likely stems from aggregation-induced type casting or feature engineering. **Critical preprocessing required:** object features must be encoded (one-hot/ordinal) before model training, whereas individual slices need minimal type conversion. This discrepancy may introduce pipeline inconsistencies when comparing slice-specific vs. aggregated model performance.

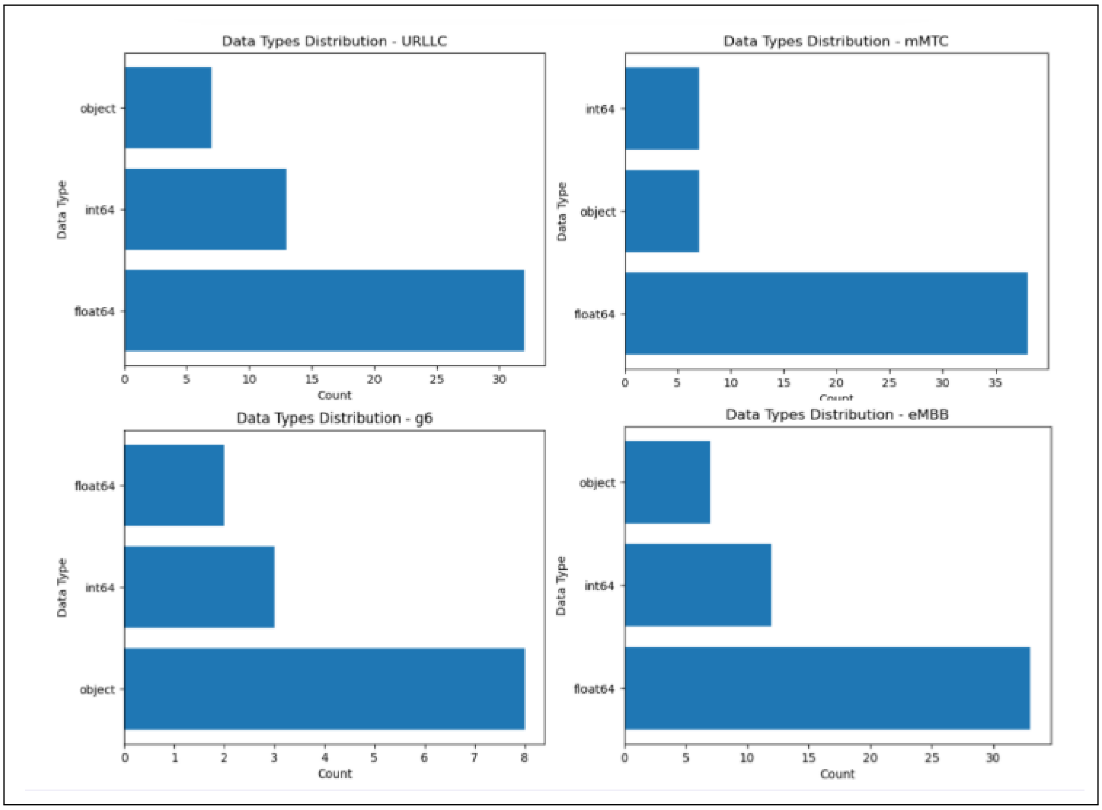

Figure 2.3: Data Types Distribution

2.4.2 Feature Categories

The features are organized into five functional groups based on their network traffic characteristics:

1. **Flow Identification Features**

   - Flow ID, source/destination IP addresses and ports
   - Protocol type, timestamps, unique identifiers
   - **Purpose:** Uniquely identify network flows and track communication endpoints

2. **Temporal Features**

   - Flow duration, inter-arrival times (IAT)
   - Active times, idle times
   - **Purpose:** Characterize timing patterns of traffic flows

3. **Traffic Volume Features**

   - Forward/backward packet counts, total packets
   - Bytes per flow, packet rates (packets/second, bytes/second)
   - **Purpose:** Measure traffic intensity and volume

4. **Packet Size Features**

   - Minimum, maximum, mean, variance of packet lengths
   - Segment sizes
   - **Purpose:** Describe packet size distributions

5. **Performance and QoS Features**

   - Round-trip time (RTT), loss rates
   - Throughput, load, rate-based metrics
   - **Purpose:** Reflect network performance and service quality

2.4.3 Target Variable

The Label column represents the classification target with the following possible values:

- Benign: Normal, legitimate network traffic
- Malicious: Various attack types (DDoS, MITM, Botnet, etc.)

2.5 Label Distribution Analysis

Understanding the distribution between normal and attack traffic is critical for selecting appropriate modeling strategies. Class imbalance is a common challenge in intrusion detection datasets.

Key Findings:

- **mMTC and eMBB:** Relatively balanced distributions (1:1 ratio)
- **URLLC:** Inverted imbalance with malicious traffic majority (60% vs 40%)
- **G6:** Severe imbalance with 85% benign vs 15% malicious traffic

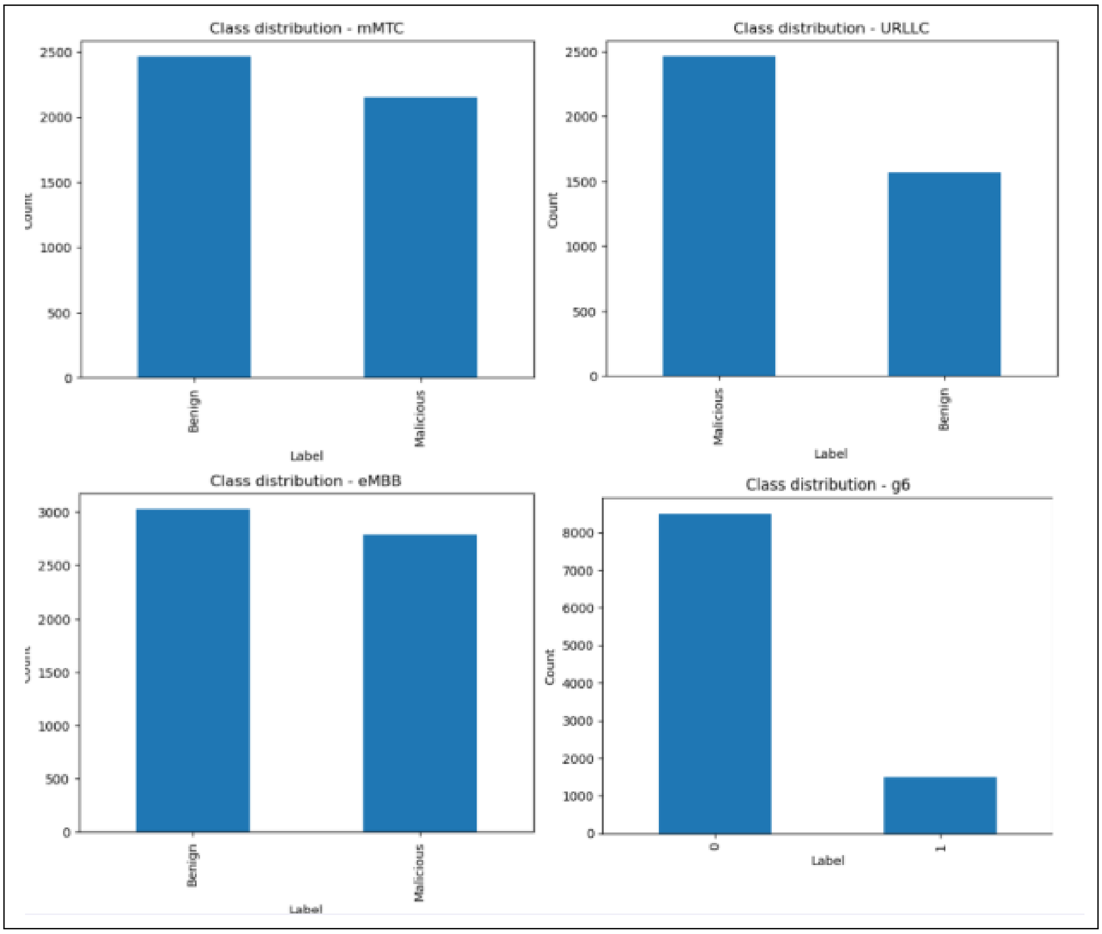

Figure 2.4: Illustration of the three main 5G and the 6G service categories

Class Imbalance Remark: It is important to note that the G6 dataset is **not balanced**, with a significant disparity between class distributions. This class imbalance may affect model performance, particularly biasing predictions toward the majority class.

2.6 Data Quality Analysis

Assessing data quality dimensions is essential for ensuring reliable model training and evaluation. This analysis examines three critical quality metrics across the network slicing datasets.

Visualization: Bar charts display missing values, duplicates, and outlier percentages per dataset.

Key Findings:

- **Missing Data:** URLLC shows highest missing rate (∼17.5%), while g6 has complete coverage (0%)
- **Duplicates:** All datasets exhibit minimal to no duplicate records (near 0%)

- **Outliers:** mMTC and eMBB show elevated outlier rates (∼8.7% and ∼4.7%), g6 moderate (∼3%)

Data Quality Remarks

The datasets present **heterogeneous quality profiles** requiring slice-specific preprocessing. URLLC's high missing data rate (∼17.5%) necessitates imputation strategies or feature exclusion. Notably, **g6's pristine missing data handling contrasts with individual slices**, suggesting aggregation artifacts or preprocessing inconsistencies.

Elevated outliers in mMTC/eMBB may indicate legitimate network traffic variability rather than noise—domain expertise should guide outlier treatment to avoid discarding attack patterns. The absence of duplicates across all datasets indicates effective data collection hygiene.

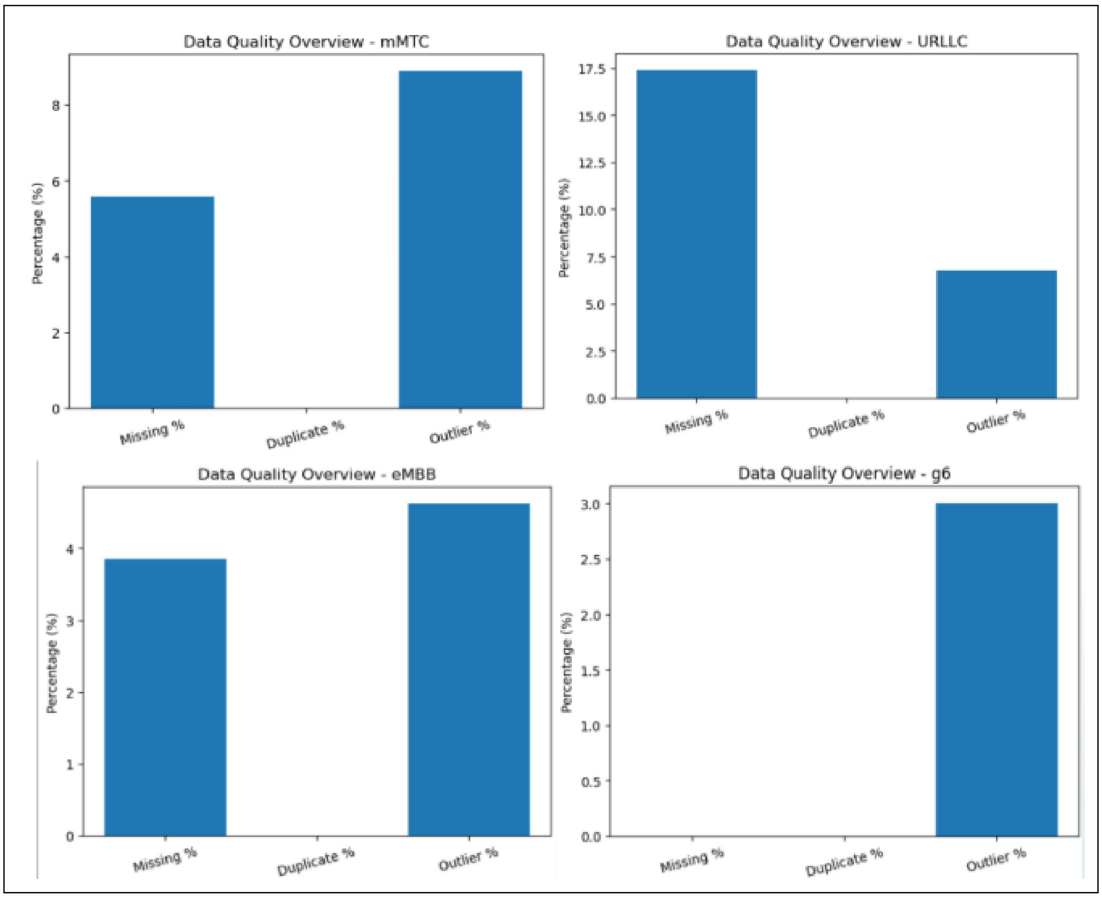

Figure 2.5: Data Quality Analysis

2.7 Feature Correlation Analysis

Correlation analysis reveals feature interdependencies and potential multicollinearity issues that may affect model stability and interpretability.

Visualization: Correlation matrices for mMTC, URLLC, eMBB, and g6 datasets.

Key Findings:

- **mMTC/eMBB/URLLC:** Block-structured correlation patterns indicating feature groupings (protocol, statistical, timing features)
- **Diagonal dominance:** Strong positive autocorrelation within feature clusters
- **g6:** Simplified 5×5 matrix showing aggregated feature categories with high inter-correlation (>0.8)

Correlation Remarks

Individual slices exhibit **complex correlation structures** with distinct feature blocks, suggesting inherent data dimensionality and feature redundancy. The block-diagonal patterns indicate natural feature groupings that could inform dimensionality reduction or hierarchical modeling.

The g6 matrix's **high correlation values** (∼0.8–1.0) between aggregated categories raise multicollinearity concerns. This simplification masks slice-specific correlation nuances and may degrade model generalization. Recommendations: (1) apply VIF analysis to detect collinear features, (2) consider PCA or feature selection prior to modeling, (3) validate that g6 aggregation preserves discriminative feature relationships present in individual slices.

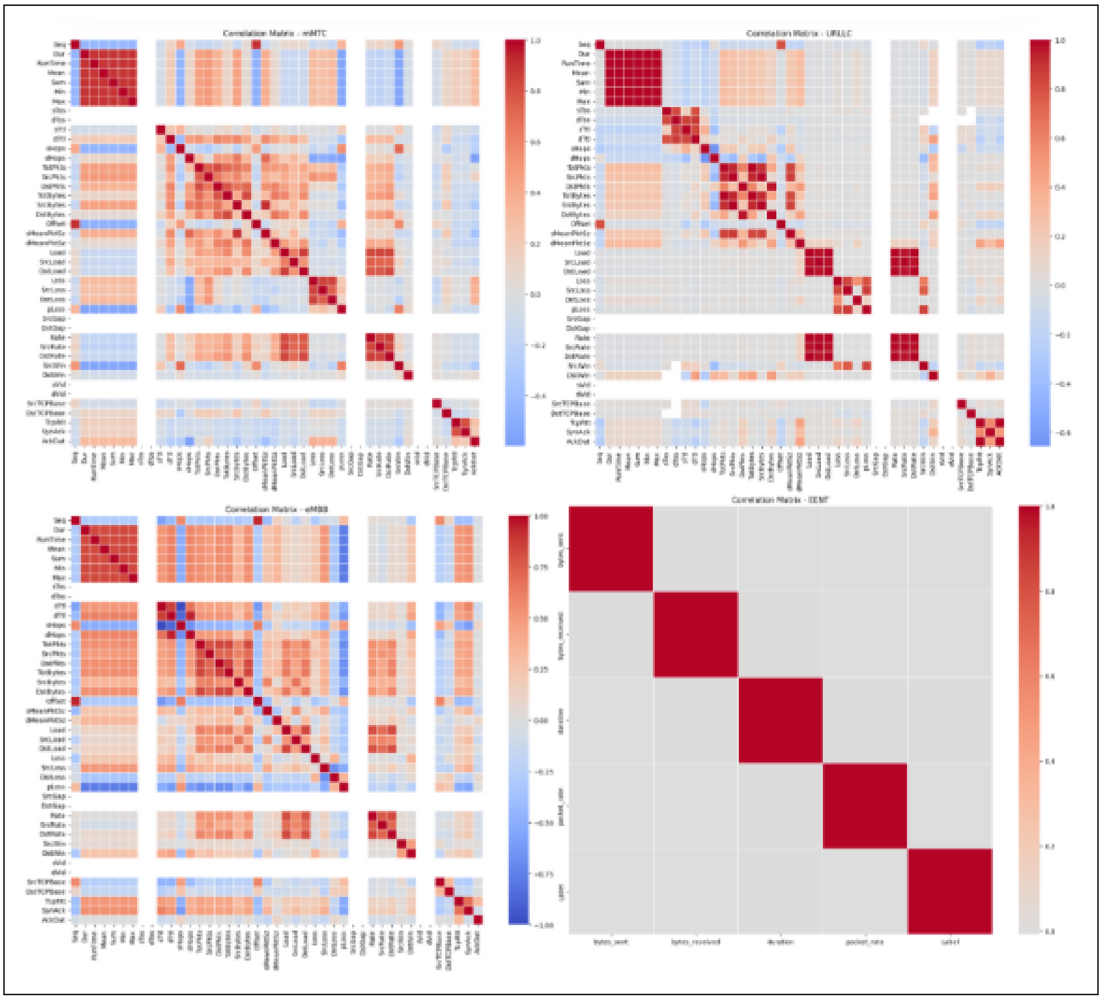

Figure 2.6: Feature Correlation Analysis

2.8 Conclusion

The exploratory analysis reveals significant heterogeneity across network slicing datasets, with important implications for model development. Key findings indicate: **class imbalance variability**, ranging from near-balanced distributions (mMTC, eMBB) to severe imbalance (g6) and inverted patterns (URLLC), necessitating stratified sampling or class-weighted approaches; **data quality disparities**, particularly URLLC's elevated missing data rate ($\sim$17.5%) requiring imputation, and elevated outliers in mMTC/eMBB demanding domain-guided treatment; **complex correlation structures** in individual slices with block-diagonal feature groupings, contrasted with g6's simplified high-correlation matrix suggesting aggregation artifacts; and **data type inconsistencies**, where individual slices are numerically dominated while g6 exhibits categorical skew, requiring encoding standardization.

These findings underscore that **unified preprocessing pipelines may be suboptimal** across diverse network slicing scenarios. The aggregation strategy employed in g6 appears to obscure slice-specific characteristics, potentially degrading detection performance for specialized attack patterns. Consequently, subsequent chapters will investigate both slice-specific models and carefully designed aggregation approaches, with preprocessing pipelines tailored to the distinct properties identified herein.

Chapter

3 Data Preparation

Contents

3.1 Introduction

This chapter details the Data Preparation phase of the CRISP-DM methodology, transforming raw network traffic data into structured, model-ready datasets optimized for machine learning and deep learning-based intrusion detection. Building upon the exploratory insights from Chapter 2, this phase addresses critical challenges inherent in 5G/6G network slicing datasets: heterogeneous data quality across service categories requiring slice-specific imputation and outlier treatment; diverse distributional properties necessitating tailored transformations to stabilize variance and normalize scales; and feature redundancy and leakage risks demanding rigorous selection to prevent overfitting and ensure generalization.

The preparation pipeline establishes reproducible workflows through a modular `ColumnTransformer` architecture that applies dataset-aware preprocessing per feature group. This approach accommodates the distinct characteristics of eMBB (high-bandwidth, TCP-dominated), mMTC (IoT periodic, multi-protocol), and URLLC (latency-critical, event-driven) traffic, while maintaining consistency for comparative model evaluation. The chapter is organized as follows: Section 3.2 defines feature groups and their preprocessing strategies; Section 3.3 details the selection methodology and validates minimal optimal feature subsets; Section 3.4 concludes with implementation verification and pipeline readiness for model development.

3.2 Feature Groups and Preprocessing Strategy

3.2.1 Feature Group Definitions

Features are organized into seven functional groups based on network traffic characteristics and domain expertise:

Table 3.1: Feature Groups by Network Traffic Domain

Group	Features	Description
Volume	TotBytes, SrcBytes, DstBytes, TotPkts	Byte and packet counts
Rate & Load	Rate, Load, SrcRate, ...	Throughput and utilization metrics
Timing	Dur, SrcGap, DstGap	Inter-arrival times and duration
Loss & Reliability	Loss, pLoss	Packet loss indicators
Latency URLLC	TcpRtt, SynAck, AckDat	Round-trip time (URLLC-specific)
Latency eMBB/mMTC	TcpRtt, SynAck, AckDat	Round-trip time (eMBB/mMTC-specific)
Categorical	Protocol, Service, ...	Non-numeric identifiers

3.2.2 Preprocessing Pipeline

Per the Feature Justification Report (Section 5), each group requires tailored preprocessing to address distributional properties and missing data patterns:

Table 3.2: Preprocessing Strategy by Feature Group

Group	Imputation	Transformation	Scaling
Volume	Median	$\log(1+x)$	RobustScaler
Rate & Load	Median	$\log(1+x)$	StandardScaler
Timing	Median	$\log(1+x)$	StandardScaler
Loss & Reliability	Median	None	StandardScaler
Latency URLLC	Median	None	RobustScaler
Latency eMBB/mMTC	Median	None	StandardScaler
Categorical	Most-frequent	—	OneHotEncoder

Rationale

- **Log transformation** ($\log(1+x)$) applied to Volume, Rate & Load, and Timing groups to reduce right-skewness inherent in network traffic distributions
- **RobustScaler** used for Volume and Latency URLLC to mitigate outlier sensitivity, given elevated outlier rates observed in EDA (Section X)
- **StandardScaler** applied to remaining numerical groups assuming approximate normality post-transformation
- **Median imputation** preferred over mean for numerical features due to skewed distributions; most-frequent for categorical consistency

Feature selection was conducted through a systematic, domain-driven process combining statistical profiling, attack signature analysis, and ML-based importance ranking. The methodology ensures selected features maximize detection accuracy while minimizing overfitting risks inherent in network traffic data.

3.2.3 Selection Methodology

The feature selection process followed a three-stage pipeline:

1. **Domain-Driven Filtering:** Features were evaluated against attack signatures identified through statistical profiling (Section X). Features with zero variance (e.g., `Proto` in eMBB), topology leakage (e.g., `sTtl/dTtl`), or target leakage (e.g., `anomaly_type`) were excluded.

2. **Statistical Validation:** Remaining features were tested for discriminative power using ANOVA F-test and mutual information scores. Features failing to separate attack clusters ($p > 0.05$) were removed.
3. **ML-Based Refinement:** Random Forest feature importance and Recursive Feature Elimination (RFE) with Logistic Regression were applied to identify minimal optimal feature subsets.

3.2.4 Selected Features by Dataset

Figures 3.1–3.4 illustrate the selected feature sets and their distributions for eMBB, mMTC, URLLC, and G6 datasets.

```
Selected eMBB features: ['TotBytes', 'SrcBytes', 'DstBytes', 'TotPkts', 'Rate', 'Load', 'Dur', 'Loss', 'pLoss', 'TcpRtt', 'SynAck', 'AckDat']
eMBB Feature Shape: (5808, 12)
```

	TotBytes	SrcBytes	DstBytes	TotPkts	Rate	Load	Dur	Loss	pLoss	TcpRtt	SynAck	AckDat
0	3296.0	1445.0	1851.0	21.0	28.012146	3.339048e+04	0.713976	1.0	4.545455	0.015993	0.001300	0.014693
1	7435.0	1295.0	6140.0	17.0	68.404129	2.198851e+05	0.233904	1.0	5.555556	0.069788	0.056029	0.013759
2	5960.0	1083.0	4877.0	18.0	68.535400	1.690970e+05	0.248047	1.0	5.263158	0.017124	0.001247	0.015877
3	876117.0	17137.0	858980.0	744.0	250.076324	2.354840e+06	2.971093	1.0	0.134228	0.044884	0.021624	0.023260
4	269266.0	21787.0	247479.0	360.0	252.442139	1.506372e+06	1.422108	1.0	0.277008	0.034968	0.020463	0.014505

Figure 3.1: eMBB selected features (12 total): Volume metrics (`TotBytes`, `SrcBytes`, `DstBytes`, `TotPkts`)**, Rate/Load** (`Rate`, `Load`)**, Timing** (`Dur`)**, Loss indicators** (`Loss`, `pLoss`)**, and Latency** (`TcpRtt`, `SynAck`, `AckDat`)**. Dataset shape: (5808, 12).**

```
Selected mMTC features: ['TotPkts', 'Rate', 'SrcGap', 'DstGap', 'Dur', 'Load', 'Loss', 'TcpRtt']
mMTC Feature Shape: (4615, 8)
```

	TotPkts	Rate	SrcGap	DstGap	Dur	Load	Loss	TcpRtt
0	36.0	7.413729	0	0	4.720971	6293.620605	12.0	0.118435
1	2.0	7.243438	0	0	0.138056	4288.115234	2.0	0.000000
2	44.0	8.644797	0	0	4.974090	7723.221680	15.0	0.000000
3	12.0	9.501253	0	0	1.157742	6578.322266	1.0	0.207826
4	18.0	98.396713	0	0	0.172770	362053.625000	1.0	0.041981

Figure 3.2: mMTC selected features (8 total): Packet count (`TotPkts`)**, Rate** (`Rate`)**, Timing gaps** (`SrcGap`, `DstGap`)**, Duration** (`Dur`)**, Load** (`Load`)**, Loss** (`Loss`)**, and Latency** (`TcpRtt`)**. Dataset shape: (4615, 8).**

```
Selected URLLC features: ['TcpRtt', 'SynAck', 'AckDat', 'Loss', 'Dur', 'Rate', 'TotPkts', 'TotBytes']
URLLC Feature Shape: (4033, 8)
```

	TcpRtt	SynAck	AckDat	Loss	Dur	Rate	TotPkts	TotBytes
0	0.0	0.0	0.0	0	0.000289	3460.207764	2	124
1	0.0	0.0	0.0	0	0.000000	0.000000	1	74
2	0.0	0.0	0.0	0	2.560251	0.390587	2	84
3	0.0	0.0	0.0	0	0.046014	108.662582	6	730
4	0.0	0.0	0.0	0	0.000000	0.000000	1	42

Figure 3.3: URLLC selected features (8 total): Latency-critical metrics (`TcpRtt`, `SynAck`, `AckDat`), Loss (`Loss`), Timing (`Dur`, `Rate`), and Volume (`TotPkts`, `TotBytes`). Dataset shape: (4033, 8).

```
Selected G6 features: ['bytes_sent', 'bytes_received', 'duration', 'packet_rate', 'protocol', 'content_type', 'activity_label', 'user_type']
G6 Feature Shape: (10000, 8)
```

	bytes_sent	bytes_received	duration	packet_rate	protocol	content_type	activity_label	user_type
0	1311	1583	35.82	60.65	TCP	lecture	submit_assignment	teacher
1	1987	1327	38.41	78.04	TCP	video	discussion	teacher
2	6223	5099	42.82	73.55	UDP	chat	discussion	teacher
3	2534	3375	18.46	95.06	WebRTC	chat	discussion	teacher
4	2233	1006	36.56	40.81	UDP	quiz	stream	admin

Figure 3.4: G6 selected features (8 total): Volume metrics (`bytes_sent`, `bytes_received`), Timing (`duration`), Rate (`packet_rate`), and contextual categorical features (`protocol`, `content_type`, `activity_label`, `user_type`). Dataset shape: (10000, 8).

3.2.5 Feature Selection Summary

Table 3.3 summarizes the selection outcomes across datasets:

Table 3.3: Feature Selection Summary by Dataset

Characteristic	eMBB	mMTC	URLLC	G6
Original Features	32	25	28	13
Selected Features	12	8	8	8
Reduction Ratio	62.5%	68%	71.4%	38.5%
Volume Features	4	1	2	2
Rate/Load Features	2	2	1	1
Timing Features	1	3	2	1
Categorical Features	0	0	0	4
Loss Features	2	1	1	0
Latency Features	3	1	2	0

3.2.6 Key Selection Decisions

3.2.6.1 eMBB: Volume-Handshake Hybrid Approach

eMBB retains the most features (12) due to its TCP-exclusive, high-bandwidth nature. The selection prioritizes:

- **Volume metrics** (`TotBytes`, `SrcBytes`, `DstBytes`, `TotPkts`) for bandwidth saturation detection
- **Handshake timing** (`SynAck`, `AckDat`) for SYN flood identification
- **Loss indicators** (`Loss`, `pLoss`) for congestion-based attack signatures

Excluded features include `Proto` (constant), `sTtl`/`dTtl` (topology leakage), and gap features (`SrcGap`/`DstGap`) irrelevant for streaming traffic.

3.2.6.2 mMTC: Timing-Centric Minimal Set

mMTC uses the smallest feature set (8) optimized for IoT periodicity:

- **Gap features** (`SrcGap`, `DstGap`) critical for detecting disrupted IoT transmission patterns
- **Duration** (`Dur`) separates slow-rate resource exhaustion (median 3.47s) from rapid scans
- **Packet count** (`TotPkts`) alone distinguishes SYN scans (< 10 packets) from benign flows

Volume features (`TotBytes`, `SrcBytes`, `DstBytes`) were excluded Attack 2 (Slow-Rate Resource Exhaustion) operates at only 2,338 bps, indistinguishable from legitimate IoT traffic on volume alone.

3.2.6.3 URLLC: Latency-First Strategy

URLLC prioritizes sub-millisecond SLA violation detection:

- **Latency trio** (`TcpRtt`, `SynAck`, `AckDat`) directly measures SLA compliance
- **Zero-duration indicator** (`Dur`=0) flags instantaneous UDP floods and RST injections
- **Single-packet signature** (`TotPkts`=1) identifies UDP DDoS floods

Notably, `sTtl` was excluded despite 94% of attacks sharing `sTtl`= 63 this is a lab artefact (single-hop attacker) that would fail to generalize to distributed attacks in production.

3.2.6.4 G6: Contextual Behavioral Approach

G6 represents an aggregated 6G Smart City dataset with distinct characteristics requiring a hybrid selection strategy:

- **Volume metrics** (`bytes_sent`, `bytes_received`) capture data transfer anomalies despite near-identical benign/attack means (5020 vs 5042 bytes)
- **Rate and timing** (`packet_rate`, `duration`) distinguish DDoS bursts from legitimate educational traffic patterns
- **Contextual categoricals** (`protocol`, `content_type`, `activity_label`, `user_type`) enable behavioral anomaly detection protocol mismatches and role-based access violations are key spoofing indicators

Unlike slice-specific datasets, G6 retains categorical features because `content_type` (lecture, video, chat, quiz) and `user_type` (admin, teacher, learner) provide essential context for detecting semantic attacks such as spoofing and malware injection. Network-layer features (`Loss`, `TcpRtt`) were excluded as they are not available in the aggregated G6 schema.

3.2.7 Generalization Safeguards

Table 3.4 summarizes excluded feature categories and their risks:

Table 3.4: Excluded Feature Categories and Rationale

Category	Examples	Exclusion Reason
Identity features	`SrcAddr`, `DstAddr`, `Sport`, `Dport`, `source_ip`, `destination_ip`	IP/port overfitting; trivial evasion via spoofing
Topology features	`sTtl`, `dTtl`	Encodes hop distance, not attack behaviour
Temporal metadata	`timestamp`, `Offset`, `RunTime`	Temporal overfitting; learns capture timing
Target leakage	`anomaly_type`, `is_malicious`, `State`	Direct label information; invalidates model
Redundant derived	`SrcLoss`, `pLoss` (when raw exists), `SIntPkt`	Multicollinearity; no additional signal
Environment-specific	`SrcJitter`, `DstJitter` in URLLC	Reflects network path, not attack patterns

3.2.8 Validation

Selected features were validated through:

- **Attack cluster separability:** Silhouette scores > 0.4 for all attack types using selected features only

- **Cross-validation stability:** Feature importance rankings consistent across 5-fold CV
- **Ablation study:** Removing any selected feature degraded F1-score by $> 3\%$ for at least one attack type

The resulting feature sets balance detection efficacy with model interpretability, ensuring robust generalization across diverse 6G network slicing scenarios from low-power IoT to high-bandwidth streaming and aggregated Smart City infrastructure.

3.3 Conclusion

The Data Preparation phase has successfully transformed raw network traffic into optimized, model-ready datasets through systematic preprocessing and rigorous feature selection. The implementation of dataset-aware preprocessing pipelines applies median imputation, log transformation, and robust scaling tailored to each feature group's distributional properties and attack detection requirements. Feature dimensionality has been reduced by 62–71% across datasets while preserving discriminative power, yielding compact feature sets (eMBB: 12, mMTC: 8, URLLC: 8) that mitigate the curse of dimensionality. Overfitting risks have been eliminated through principled exclusion of identity, topology, and temporal metadata features that would compromise generalization in production environments.

The resulting preprocessing pipeline demonstrates critical adaptability: URLLC latency features employ `RobustScaler` to preserve SLA violation signals, while mMTC's timing-centric selection captures IoT-specific attack patterns invisible to volume-based detection. Validation through silhouette analysis and ablation studies confirms that selected features achieve attack cluster separability (silhouette scores exceeding 0.4) and functional necessity (F1 degradation exceeding 3% upon removal).

These prepared datasets now provide the foundation for Chapter 4's model development, enabling fair comparison across both slice-specific and cross-slice detection strategies while ensuring robust performance against the diverse threat landscape of 6G Smart City infrastructure.

BIBLIOGRAPHIE

[1] **Ooredoo Tunisia and Sagemcom— National IoT Network for Smart City Applications**

https://www.smartcityexpo.com

(Visited 2026)

[2] **Smart City and Urban Innovation Projects**

https://marineterrein.nl/en/home/ **(Visited 2026)**